Practical-3-0s-2520090235

Task1:

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    printf("Parent Process Started\n");
    printf("Parent PID  : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());
    pid=fork();

    if (pid<0) {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid==0) {
        printf("Child Process\n ");
        printf("Child PID  : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());
        printf("Child State: Running\n");
        sleep(5);

        printf("Child State: Terminating\n");
        exit(0);
    }
    else {
        printf("Parent Process\n ");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        printf("Parent State: Running\n");
        sleep(2);
        printf("Parent State: Waiting for child\n");
        wait(NULL);

        printf("Parent State: Child Terminated\n");
    }
    return 0;
}
```

Output:

```
Parent Process Started
Parent PID  : 569
Parent PPID : 338
Parent Process
 Parent PID : 569
Child PID  : 570
Child Process
Parent State: Running
 Child PID  : 570
Child PPID : 569
Child State: Running
Parent State: Waiting for child
```

Observation: The program creates parent and child processes using `fork()` and displays their PID and PPID. The parent waits for the child to complete before terminating.

Task2: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());
        sleep(10);
        printf("Child process terminated.\n");
    }
    else {
        printf("Parent process started. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        wait(NULL);
        printf("Parent: Child terminated.\n");
    }
    return 0;
}
```

Output

```
Parent process started. PID = 1036
Child process Started. PID= 1037
Child PID = 1037
Child process terminated.
Parent: Child terminated.
```

Observation:

The Linux commands `ps`, `top`, and `/proc` successfully show the process IDs and their states. The experiment demonstrates transitions between **Running, Waiting, and Terminated** states during process execution.